PRADITA University

Concepts, Organization, and a Case Study

Agentic SDLC

Six Agents, One User Story

October 9, 2026

Alfa Yohannis



Contents

PRADITA University

1 Goals
2 Agentic SDLC in General
3 Organizing an Agent Team
4 A Test-First Process
5 The Case Study
6 Architecture
7 Lessons and Open Questions
8 Summary

PRADITA University

What happens when every role on the team is held by an agent?

PRADITA University

Goals

Learning Goals

PRADITA University

- Explain the agentic SDLC cycle and its test-first variant: the phases agents carry out, the gates people keep, and the feedback loops.
- Describe how a team of agents is organized and coordinated: roles, ownership, layers of context, and orchestration or choreography.
- Reason about autonomy and control: what agents may do on their own, and what needs a person.

PRADITA University

Agentic SDLC in General

From SDLC to Agentic SDLC

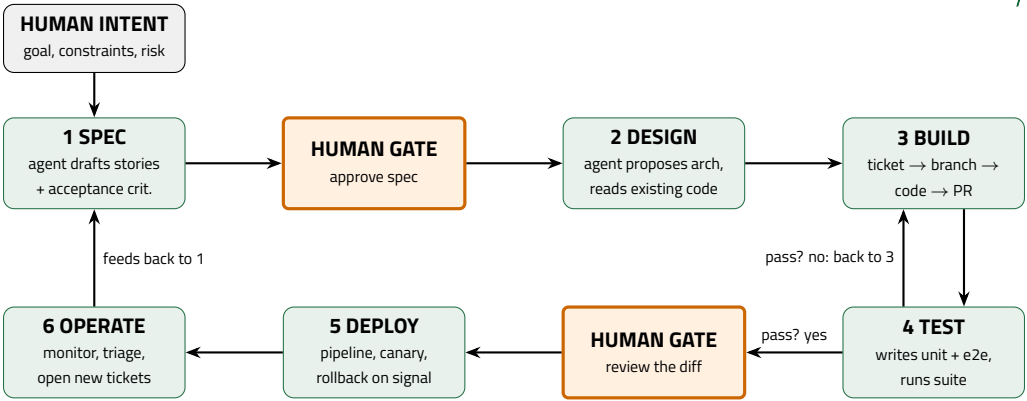
PRADITA University

- The software development life cycle (SDLC) splits building software into phases: specification, design, build, test, deployment, and operation.
- In an agentic SDLC the phases stay, but AI agents do the work inside them. They draft user stories, propose an architecture, write code and tests, and run the pipeline.
- People set the goal, the constraints, and the risk at the start, and they hold two decision gates in the middle of the cycle.

The gates keep the responsibility for what is built and released with people, however fast the agents work.

The Agentic SDLC Cycle

PRADITA University



Green boxes are agent work. Orange boxes are human decisions.

Human Gates

PRADITA University

- The first gate approves the specification. Before any design or code exists, a person checks that the user stories and acceptance criteria say what they meant. A misunderstanding is cheapest to fix here.
- The second gate reviews the diff. Before release, a person reads the changes that passed the tests, because passing tests only prove the behavior they test.
- Between the two gates, agents may loop on their own: write code, run the tests, and fix what fails without waiting for anyone.

Two Feedback Loops

PRADITA University

- Inner loop: when a test fails, the work goes back to BUILD. Agents run this loop themselves, as often as it takes.
- Outer loop: OPERATE watches the running application and triages the problems it finds, which means sorting them by impact and urgency, setting duplicates and noise aside, and deciding who handles each one. The result is new tickets that go back into SPEC.

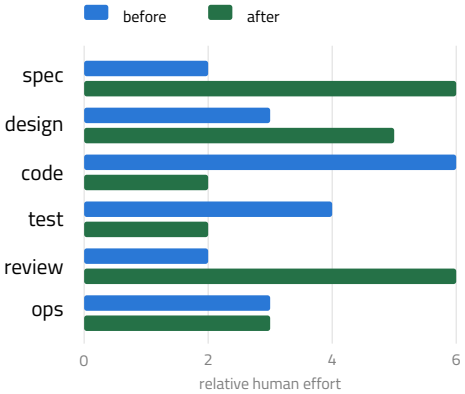
The inner loop keeps the code in line with the specification. The outer loop keeps the specification in line with what happens in production.

Who Does What

PRADITA University

Phase	Agent	Human
SPEC	drafts user stories and acceptance criteria	states the goal, constraints, and risk, then approves the specification
DESIGN	proposes an architecture after reading the existing code	-
BUILD	ticket, branch, code, then a pull request	-
TEST	writes unit and end-to-end tests and runs the suite	-
DEPLOY	pipeline, canary release, rollback on a signal	reviews the diff before release
OPERATE	monitors, triages problems by impact and urgency, opens new tickets	-

Where Human Effort Goes



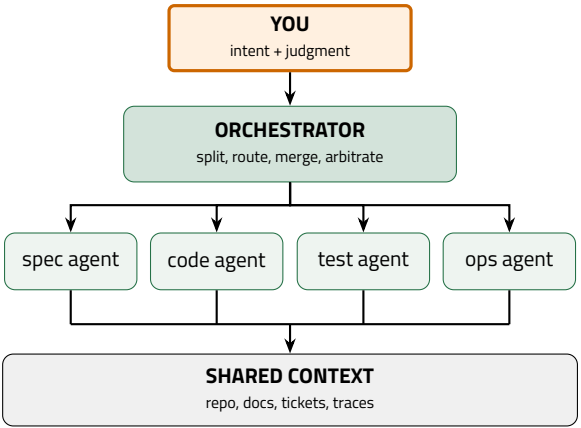
- Less effort on code (from 6 to 2) and tests (from 4 to 2).
- More effort on the specification (from 2 to 6), design (from 3 to 5), and review (from 2 to 6).
- Operations stays at 3.

People spend their time saying what to build and checking what was built. Agents do most of the building in between.

PRADITA University

Organizing an Agent Team

One Orchestrator, Many Agents

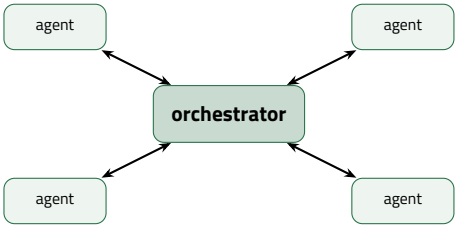


- The orchestrator splits a goal into tasks, routes each task to an agent, merges the results, and decides when agents disagree.
- Every agent works from the same shared context: the repository, the docs, the tickets, and the traces of earlier runs.
- The person talks to the orchestrator and keeps the final judgment.

Orchestration and Choreography

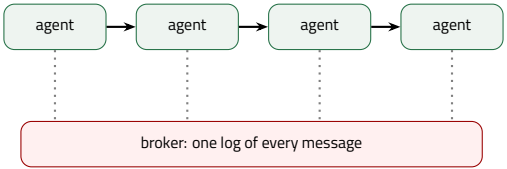
PRADITA University

Orchestration



one agent decides who works next

Choreography



each agent hands work on by shared rules

An orchestrator gives one point of control and one point of failure. Choreography spreads the decisions over the agents, so the shared rules and the message log have to carry the coordination. This case study uses choreography.

Choosing a Coordination Pattern

PRADITA University

	Orchestration	**Choreography**
Who decides the next step	the orchestrator	each agent, from the team's rules
Control and failure	one central point	spread over the agents
Adding an agent	change the orchestrator	add the agent and update the rules
Where to see what happened	the orchestrator's records	the message log
Suits	work that must be split and rebalanced as it goes	a workflow that is known in advance

Roles and Ownership

PRADITA University

- Each agent has one role and owns a set of artifacts. Alice owns the specification, Bob the tests, Charlie the database, Frank the back end, David the front end, and Erin the documentation.
- Every agent may read everything, but it changes only what it owns. A change to another artifact goes to its owner as a message.
- Ownership keeps two agents from editing the same file at once, and it gives a clear answer to the question of who is responsible.

This is code ownership, as human teams practise it, applied to agents.

Layers of Context

PRADITA University

Team context goal, team, workflow, and rules	all agents, written before the run
Role context who I am, what I own, where my limits are	one agent, written before the run
Conversation memory what I did and decided earlier	one agent, grows during the run
Task message what to do now, and who asks	one turn
Working material specification, tests, code	read when needed

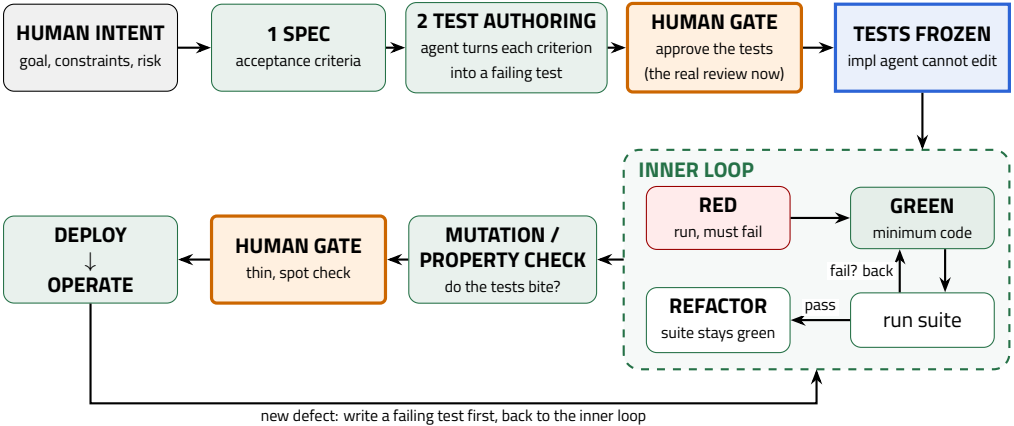
An agent knows only what these layers tell it. Anything the team depends on has to be written into one of them.

PRADITA University

A Test-First Process

A Test-First Variant of the Cycle

PRADITA University



What Changes in the Test-First Variant

PRADITA University

- The human review moves to the tests. Once a person approves them, the tests are the specification the code has to meet.
- The tests are frozen, so the implementing agent cannot make a failing test pass by editing it.
- In the inner loop the agent goes red, green, refactor: the new test fails, the smallest change makes it pass, and refactoring keeps the suite green.
- A mutation or property check tests the tests: change the code on purpose, and some test should fail.
- The last human gate shrinks to a spot check. A new defect starts as a failing test.

Tests as Contracts Between Agents

PRADITA University

- When the tests are written first, they fix the interfaces before any code exists: the data functions, the login API, and the elements on the page.
- With the interfaces fixed, the database, back-end, and front-end agents can work at the same time without waiting for each other.
- A failing test tells its owner exactly what is missing. A dispute about a test goes back to the agent who owns the tests.

In one run, Frank noticed that a test contradicted the specification and asked Bob to change it. Bob agreed and fixed the test before Frank went on.

Handoffs and Definitions of Done

PRADITA University

- A handoff is an explicit message: what is ready, where it is, and what the receiver needs to know.
- Each role has its own definition of done. The specification is done when it can be tested. The tests are done when they fail for the right reason. The code is done when its tests pass. The docs are done when they match the code.
- Erin starts only after three handoffs have arrived, which makes her step a join point in the process.

A reply arrives as a new message, so no agent sits blocked while it waits for an answer.

PRADITA University

The Case Study

The User Story: A Login Page

PRADITA University

- The user logs in with an email address, and its format is checked.
- The password has at least three words and at least two spaces, for example “green apple house”.
- A button shows and hides the password.
- Errors come with messages a user understands.
- Three failed attempts lock the login for five seconds, and the page tells the user to wait.

Written by a person acting as product owner. Everything after this point is done by the agents.

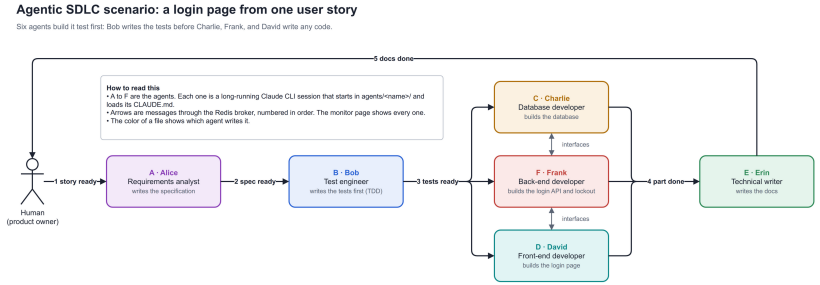
Six Roles

PRADITA University

Agent	Name	Role	Owns
A	**Alice**	Requirements analyst	the specification
B	**Bob**	Test engineer, tests first	the tests and the test contract
C	**Charlie**	Database developer	the database layer
D	**David**	Front-end developer	the login page
E	**Erin**	Technical writer	the documentation
F	**Frank**	Back-end developer	the login API and the lockout rule

The source code of the whole setup is at
`github.com/alfa-yohannis/agentic-sdlc`.

Workflow


Agentic SDLC scenario: a login page from one user story
Six agents build it test first: Bob writes the tests before Charlie, Frank, and David write any code.
5 docs done
How to read this
• A to F are the agents. Each one is a long-running Claude CLI session that starts in agents/<name>/ and loads its CLAUDE.md.
• Arrows are messages through the Redis broker, numbered in order. The monitor page shows every one.
• The color of a file shows which agent writes it.
Human
(product owner)
1 story ready
A · Alice
Requirements analyst
writes the specification
2 spec ready
B · Bob
Test engineer
writes the tests first (TDD)
3 tests ready
C · Charlie
Database developer
builds the database
interfaces
F · Frank
Back-end developer
builds the login API and lockout
interfaces
D · David
Front-end developer
builds the login page
4 part done
E · Erin
Technical writer
writes the docs
agents/shared_context/ read by every agent
user-story-login.md
the user story
by the human
requirements-login.md
the specification
by Alice
test-contract.md
interfaces the tests expect
by Bob
team.md
team, workflow, message rules
by the human
scenario.drawio
this diagram
by the human
Every agent gets team.md in its context, next to its own CLAUDE.md.
target_app/ the product, including its docs
tests/
unit, integration, and e2e tests
by Bob
database/
SQLite schema and data access
by Charlie
backend/
FastAPI app, login API, lockout
by Frank
frontend/
the login page
by David
docs/
user and developer guides, plus README.md
by Erin
Each folder belongs to one agent. The others read it, and ask its owner when it needs a change.

What the First Run Produced

PRADITA University

Artifact	Contents
Specification	2 validation rules, 25 functional and 17 non-functional requirements, 23 acceptance criteria, and 6 open questions for the product owner
Tests	138 test functions: 49 unit, 35 integration, and 54 end-to-end
Test contract	the interfaces the tests expect from the database, the back end, and the front end
Code	database layer, back end, and login page, written against the contract

Alice turned vague phrases such as “a simple error message” into testable rules and wrote down her assumptions. What she could not decide went to the human as questions.

This Case Study in the Cycle

PRADITA University

Part	In this case study
SPEC	the human writes the user story, Alice writes the specification and its acceptance criteria
Approve spec	not in place yet: Alice hands the specification straight to Bob
DESIGN	partly: the team context fixes the tech stack, and the test contract fixes the interfaces
Test authoring	Bob turns the acceptance criteria into tests before any code exists
Tests frozen	by instruction only: the developers are told not to edit the tests
BUILD	Charlie, Frank, and David build their parts at the same time, without branches or pull requests
Orchestrator	none: the agents hand work to each other through the broker
Documentation	Erin, after the tests pass
Mutation check, deploy, operate	outside this case study

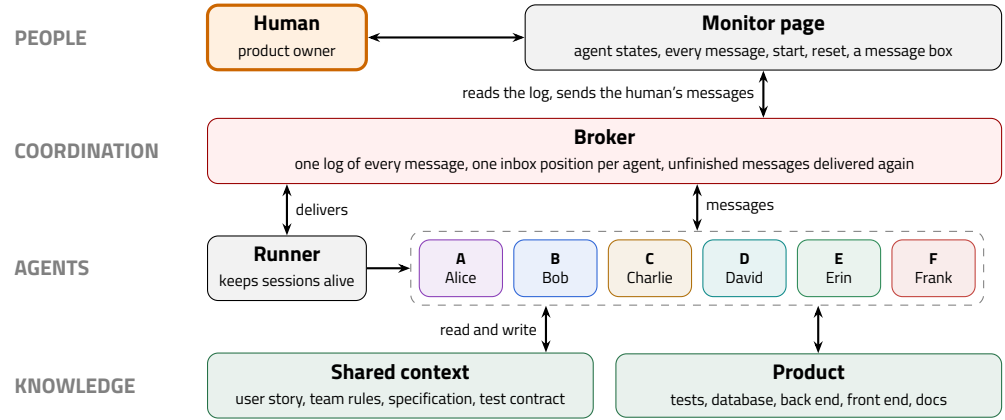
The missing parts are the next steps, for example making Alice wait for human approval before she hands the specification to Bob.

PRADITA University

Architecture

A Conceptual Architecture

PRADITA University



Components and Responsibilities

PRADITA University

Component	Responsibility
Monitor page	shows each agent's state and every message, starts and resets the team, lets the human message an agent
Broker	keeps one log of all messages, gives each agent its own position in the log, delivers again what was not finished
Runner	keeps one long-running session per agent, turns each message into a turn, restarts a session that dies
Agent session	holds one role, its context, and a conversation that continues from message to message
Shared context	the team's common knowledge: goal, rules, specification, contracts
Product	what the team builds: code, tests, documentation

Three Messaging Patterns

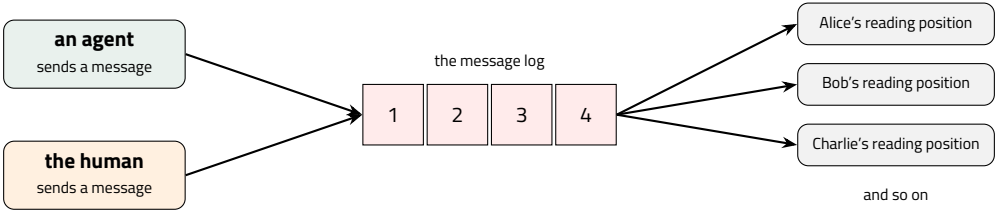
PRADITA University

	Broadcast	Queue	Log
Messages are kept	no	until taken	yes
A recipient that is down	loses the message	gets it later	gets it later
History for observers	none	gone once taken	complete
Completion is confirmed	no	no	yes

The team uses a log. One structure delivers messages to the agents and gives the monitor the complete history, and every agent keeps its own reading position in it.

One Log, One Position per Agent

PRADITA University



Every message reaches every reading position, and each agent takes only the messages addressed to it. A message counts as handled once the agent's turn on it has ended, so an interrupted turn is delivered again.

Agents as Long-Running Sessions

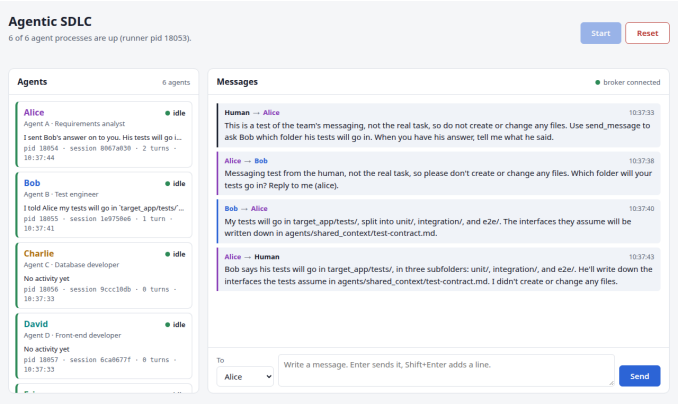
PRADITA University

- Each agent is a session that stays alive between messages, so it remembers what it did and why.
- A message becomes a new turn in the same conversation. While it waits, the session costs nothing.
- When a session dies, it is restarted with its memory, and the unfinished message is delivered again.

The simpler alternative starts a fresh session for each task, which rereads its context and forgets everything in between.

Watching the Team

PRADITA University



Left: the agents and what each one is doing. Right: every message between them, including those to and from the human.

Granting Just Enough

PRADITA University

- The tester and the developers get one narrow capability: prepare the app's environment and run its tests.
- Alice only writes the specification, so she received nothing.
- Running tests means running code, so the agents belong on a machine you trust, or in a container.

Least privilege applies to agents as it does to people and services: grant per capability, and grant only what the role needs.

PRADITA University

Lessons and Open Questions

Lessons from the Case Study

PRADITA University

- Context has to be explicit. An agent knows only what its context layers and its messages tell it.
- Tests written first work as contracts and let three agents build at the same time.
- A broker makes the collaboration visible, because every handoff and question can be read afterwards.
- Refusing by default turns a blocker into a question for a person.
- Agents follow the process they are given, so the human gates have to be designed in from the start.

Open Questions

PRADITA University

- Where should the human gates sit, and how much should each one check?
- How can tests be frozen by the system, and not only by instruction?
- When does an orchestrator help, and when is choreography enough?
- How do we know that the tests are strong enough, for example through mutation testing?
- How do the time and usage of an agent team compare with the same work done by people?

PRADITA University

Summary

Summary

PRADITA University

- An agentic SDLC keeps the SDLC phases. Agents do the work inside them, and people keep the intent and the gates.
- Human effort moves from writing code and tests to specifying and reviewing.
- An agent team needs roles, ownership, layered context, and a coordination pattern: an orchestrator, or choreography through a broker.
- A test-first process turns approved tests into the contract the code must meet, and lets agents build in parallel.
- Autonomy works inside boundaries: refuse by default, grant narrow capabilities, and keep every message visible.

PRADITA University

Source Code

`github.com/alfa-yohannis/agentic-sdlc`

The agent team, the monitor page, the documentation, and these slides.